

1 Dijkstra's

- 1.1 In an ocean, there could be multiple shortest paths between two whales. Given a Group of Whales, G , that are all equally spaced apart from each other in integer amounts and can travel these currents in either direction (which can be thought of as edges) with a starting Whale of Stella s and a target whale Tim t , devise an algorithm that returns the shortest path from s to t that uses the fewest number of edges. You can assume that there will be no ties.

Hint: Consider how you can modify the graph and run a shortest paths algorithm.

The main idea is to add a whale-fully small amount such as $1/|V|$ to each edge, and run Dijkstra's to find the shortest path from s to t . This amount is chosen to be whale-fully small enough to not affect the shortest path but this amount will bias our path to choose the one with the fewest number of edges.

Adding $1/|V|$ ensures that all paths will increase by less than 1. Since edge weights are integral (integer amounts), this means that if pathA and pathB were originally two shortest paths between s and t , $\text{floor}(\text{len}(\text{pathA})) == \text{floor}(\text{len}(\text{pathB}))$. In other words, this maintains that paths that weren't among the original shortest paths won't be shorter than those paths. However, out of the original shortest paths, the path that uses less edges will be shorter given this modification and we can whale-fully be on our way to Tim!

2 Maximal Spanning Trees

2.1 We have a graph that represents the social network of everyone in UPE, the LS CS Honor Society. The undirected edge weights correspond to an integer between 1 and 100 that states how well the 2 people (denoted by vertices) know each other! Consider the problem of finding a Maximum Spanning Tree.

- (a) Describe a modification to our graph such that Kruskal's algorithm would allow us to find a Maximum Spanning Tree of a graph.

Negate all the edge weights and find the minimum spanning tree using Kruskal's algorithm. Nothing in Kruskal's algorithm assumes the weights are positive. Therefore, the minimum of the weights negated, is achieved by the maximum of the original weights, and we will have a Maximum Spanning Tree.

Although not asked, similar logic applies for using Prim's algorithm with negated edge weights.

- (b) Can we use the same modification approach to our graph such that Dijkstra's algorithm finds the Maximum Path between two UPE members?

No, because Dijkstra's doesn't work with negative edge weights. This is because Dijkstra's relies on the assumption that if all weights are non-negative, adding an edge can never make a path shorter. Therefore, we cannot simply negate edge weights and use Dijkstra's to find the Maximum path.

3 UPE

- 3.1 You are the President of Berkeley's UPE Chapter! In order to manage your candidates, you have appointed officers to schedule 1-on-1s with candidates. Every officer can govern their candidates as well as any candidates that are friends of their candidate. Your job as President is to figure out the optimal way to allocate officers and build connections between candidates.

Formally, consider a graph G with vertices V and edges E . Each vertex v represents a candidate. It has an associated cost c , the cost of installing an officer in the town. Each edge e represents a potential connection. It has an edge weight w , the cost of building that connection (in units of time). Devise an algorithm that can efficiently compute which candidates to schedule 1-on-1s with which officers as well as which connections to form, such that every candidate in UPE has an officer that manages them (either has an officer to 1-on-1 with or is connected by friends to someone with a 1-on-1 to an officer).

We can formulate this problem as a Minimum Spanning Tree problem. We create a dummy node S . We connect S to every vertex with edge weight c , the cost of that vertex. We then find the MST of this modified graph, and that is the solution. Why does this method work? We know that the MST must include S , by definition of spanning tree. Every edge in the MST that is outgoing from S represents a selected candidate. In the MST, every vertex is either directly connected to S , i.e. a candidate with an officer, or connected to S by a series of selected edges, i.e. connected to some candidate with an officer via some connections. Since the MST finds the minimum cost solution, this is our desired arrangement of officers and connections!

4 Trie-ing to be Whale-fully efficient

- 4.1 Given a list of Professional Events (possibly repeated), devise a strategy to efficiently return a list of all the words that start with a given prefix.

Put all the Professional Events into a trie, lookup the prefix in the trie, and iterate across all the children rooted at that node.

5 Whalespaper

- 5.1 Given a dictionary of n words, describe a procedure for checking if a new compound word of length l can be created out of the concatenation of two words in the dictionary. For example, if our dictionary contains the words, "news", "paper", "new", and "whales", given a new word: "newspaper" check whether or not it is the concatenation of any two words in our dictionary. Give the runtime for your answer and explain why.

We can put all of the words in the dictionary into a trie.

Then we check all possible prefixes of the word.

If the current trie does not have a certain prefix, then we know that all other prefixes are not in the trie, thus terminating the code. (Ex: if "new" is not in the trie, then obviously "news" is not either)

If the prefix is in our dictionary, we check if the remainder of the word is also in the trie. If the remainder does not exist in the trie, we check the next prefix. If both cases are satisfied, then we have a compound word. For example, if "ne" is a word, and "wspaper" is a word, then there exists a valid pairing in our trie.

Runtime is $O(l^2)$ and $\Omega(1)$. In the worst case, you must check all l prefixes and each prefix is a valid word, meaning we also check the corresponding suffix. Checking both the prefix and suffix pair with the trie takes l operations giving us $O(l^2)$. In the best case, the first letter of the compound word does not exist in the first level of the trie. Then we can break the loop early as none of the following prefixes could possibly form a trie.